

VacPet

Relatório Técnico do Sistema

Projeto	VacPet — Carteira de Vacinação on-chain
Evento	Hackweb — Desafio 1 ProofChain Trilha Blockchain
Autor	Lielson Dos Santos Andrade
Versão	1.0.0
Rede	Ethereum Sepolia Testnet
Contrato	0x87674376e5eE07ECe76D697446Fa25617214E9F4
Data	Junho de 2026

1. Resumo Executivo

O VacPet é um sistema descentralizado de carteirinha de vacinação para animais de estimação, desenvolvido na rede Ethereum (Sepolia Testnet). O sistema resolve o problema da perda, adulteração e inacessibilidade de registros de saúde animal, utilizando a imutabilidade da blockchain para garantir que o histórico vacinal de cada animal fique disponível publicamente, de forma permanente e verificável por qualquer pessoa.

O projeto foi desenvolvido para o Hackweb, Desafio 1 da trilha ProofChain, e compreende um smart contract solidity, uma interface web standalone (HTML/JS) e uma suíte completa de testes automatizados com 100% de cobertura de código.

Indicadores-Chave

Indicador	Resultado	Observação
Testes automatizados	42 testes	100% passando
Cobertura de statements	17/17 — 100%	Todas as linhas exercitadas
Cobertura de branches	18/18 — 100%	Todos os desvios testados
Cobertura de funções	10/10 — 100%	Todas as funções cobertas
Funções de escrita (gas)	3 funções	cadastrar, registrar, transferir
Funções de leitura (grátis)	5 funções	Consultas públicas sem custo
Dados nunca armazenados on-chain	CPF, endereço físico	Privacidade preservada por design

2. Problema e Solução

2.1 Contexto e Problema

Tutores de animais de estimação dependem de carteirinhas físicas de vacinação sujeitas a perda, rasgo e esquecimento. Clínicas veterinárias mantêm registros em sistemas isolados e proprietários, sem interoperabilidade. Quando um animal é adotado, vendido ou levado a uma nova clínica, o histórico de saúde frequentemente se perde, prejudicando a continuidade do cuidado e criando riscos sanitários.

Os principais problemas identificados:

- Carteirinha física se perde ou deteriora com o tempo;
- Clínica que fecha leva consigo os registros do animal;
- Adulteração de registros vacinais não é detectável;
- Verificação do histórico exige contato com clínica específica;
- Transferência de tutor ocorre sem histórico completo do animal.

2.2 A Solução VacPet

O VacPet usa blockchain Ethereum para criar um registro imutável, público e auditável de vacinas. Cada animal recebe um ID único on-chain. Qualquer pessoa pode verificar o histórico de vacinação sem necessidade de conta, senha ou intermediário.

Problema	Como o VacPet resolve
Carteirinha física se perde	Registro permanente e imutável on-chain
Clínica fecha e perde dados	Dados não dependem de nenhum servidor centralizado
Adulteração de registros	Imutabilidade da blockchain garante integridade
Verificação depende de ligação/e-mail	Consulta pública instantânea por qualquer endereço
Transferência sem histórico	Histórico completo segue o animal na transferência

3. Arquitetura do Sistema

3.1 Visão Geral

O sistema é composto por duas camadas principais: uma interface web estática (off-chain) e um smart contract na blockchain Ethereum (on-chain). A comunicação entre as camadas é feita pela biblioteca Ethers.js, com o MetaMask intermediando a assinatura e envio de transações.

Camada Off-Chain	Camada On-Chain
Frontend HTML + CSS + JavaScript	Smart Contract Solidity ^0.8.20
Ethers.js v6 (comunicação com contrato)	Rede Ethereum — Sepolia Testnet
MetaMask (carteira e assinatura)	Armazenamento permanente no EVM
Dados sensíveis (CPF, endereço físico)	Dados públicos do animal e vacinas

3.2 Estrutura de Diretórios

```
vacpet/
├── contracts/
│   └── VacPet.sol           # Smart contract principal
├── scripts/
│   └── deploy.js           # Script de deploy na Sepolia
├── test/
│   └── Vacpet.test.js      # 42 testes automatizados
├── frontend/
│   └── index.html          # Interface web completa
├── hardhat.config.js       # Configuração Hardhat + plugins
├── package.json
└── .env.example            # Template de variáveis de ambiente
```

3.3 Dados On-Chain vs Off-Chain

Uma decisão de design crítica do VacPet é a separação explícita entre o que é armazenado na blockchain e o que permanece fora dela:

On-Chain (público e permanente)	Off-Chain (privado ou temporário)
Nome, espécie, raça, cor do animal	Interface visual (frontend HTML)
Data de nascimento do animal (timestamp)	CPF do tutor
Endereço (wallet) do tutor	Endereço físico da residência
Histórico completo de vacinas aplicadas	Dados sensíveis em geral
Nome, lote, data de cada vacina	Configuração do ambiente (.env)
CRMV e clínica do veterinário	Private key do deployer

4. Smart Contract — VacPet.sol

4.1 Visão Geral

O contrato VacPet.sol é implementado em Solidity ^0.8.20 e representa o núcleo do sistema. Não possui owner privilegiado (sem Ownable), não é upgradeable, e não depende de oráculos externos. Toda lógica é autossuficiente e transparente.

Atributo	Valor
Linguagem	Solidity ^0.8.20
Licença	MIT
Rede	Ethereum Sepolia Testnet
Endereço deployado	0x87674376e5eE07ECe76D697446Fa25617214E9F4
Padrão de acesso	Sem owner central — controle por tutoria individual
Upgradeable	Não — imutável por design

4.2 Estruturas de Dados

Struct Animal

Representa um animal cadastrado na blockchain.

Campo	Tipo	Descrição
id	uint256	Identificador único sequencial (começa em 1)
nome	string	Nome do animal
especie	string	Espécie (Cão, Gato, Coelho...)
raca	string	Raça do animal
dataNascimento	uint256	Timestamp Unix da data de nascimento
cor	string	Cor ou pelagem
tutor	address	Endereço (wallet) do tutor responsável
ativo	bool	true = cadastro ativo; false = desativado
criadoEm	uint256	Timestamp imutável do cadastro (block.timestamp)

Struct Vacina

Representa uma vacina aplicada em um animal.

Campo	Tipo	Descrição
nome	string	Nome da vacina (V10, Antirrábica, Gripe...)
lote	string	Número de lote para rastreabilidade
dataAplicacao	uint256	Timestamp Unix da data de aplicação
proximaDose	uint256	Timestamp da próxima dose (0 se não houver)
veterinario	string	Nome e CRMV do veterinário responsável
clinica	string	Nome da clínica ou hospital veterinário

4.3 Variáveis de Estado

Variável	Tipo	Propósito
_proximold	uint256 private	Contador de IDs (inicia em 1)
animais	mapping(uint256 => Animal)	Dados completos de cada animal por ID
vacinas	mapping(uint256 => Vacina[])	Histórico de vacinas por animal (append-only)
animaisPorTutor	mapping(address => uint256[])	IDs dos animais de cada tutor

Trecho do contrato — VacPet.sol

```

contract VacPet {

    // -----
    // ESTRUTURAS DE DADOS
    // -----

    /**
     * @notice Representa uma vacina aplicada em um animal.
     * @dev Todos os timestamps são Unix (segundos desde 01/01/1970).
     * `proximaDose` igual a 0 indica que não há próxima dose prevista.
     */
    struct Vacina {
        string nome; // Nome da vacina. Ex: "V10", "Antirrábica", "Gripe"
        string lote; // Número do lote da vacina para rastreabilidade
        uint256 dataAplicacao; // Timestamp Unix da data de aplicação
        uint256 proximaDose; // Timestamp Unix da próxima dose (0 se não houver)
        string veterinario; // Nome e CRMV do veterinário responsável
        string clinica; // Nome da clínica ou hospital veterinário
    }

    /**
     * @notice Representa um animal cadastrado na blockchain.
     * @dev O campo `ativo` permite desativar o registro sem apagar o histórico,
     * preservando a auditabilidade. `criadoEm` é o timestamp do bloco
     * em que o animal foi cadastrado – prova de existência on-chain.
     */
    struct Animal {
        uint256 id; // Identificador único sequencial do animal
        string nome; // Nome do animal. Ex: "Thor", "Mel"
        string especie; // Espécie. Ex: "Cão", "Gato", "Coelho"
        string raca; // Raça do animal. Ex: "Labrador", "SRD"
        uint256 dataNascimento; // Timestamp Unix da data de nascimento
        string cor; // Cor ou pelagem. Ex: "Caramelo", "Preto e branco"
        address tutor; // Endereço (wallet) do tutor responsável
        bool ativo; // true = cadastro ativo; false = desativado
        uint256 criadoEm; // Timestamp Unix do momento do cadastro (block.timestamp)
    }

    // -----
    // MODIFICADORES
    // -----

    /**
     * @notice Garante que apenas o tutor registrado do animal pode executar a função.
     * @param animalId ID do animal a ser verificado.
     */
    modifier apenasTutor(uint256 animalId) {
        require(
            animais[animalId].tutor == msg.sender,
            "VacPet: apenas o tutor pode executar esta acao"
        );
        _;
    }

    /**
     * @notice Garante que o animal com o ID informado existe e está ativo.
     * @param animalId ID do animal a ser verificado.
     */
    modifier animalExiste(uint256 animalId) {
        require(
            animais[animalId].ativo,
            "VacPet: animal nao encontrado ou inativo"
        );
        _;
    }
}

```

```
function registrarVacina(
    uint256 animalId,
    string calldata nomeVacina,
    string calldata lote,
    uint256 dataAplicacao,
    uint256 proximaDose,
    string calldata veterinario,
    string calldata clinica
) external animalExiste(animalId) apenasTutor(animalId) {

    // Adiciona a vacina ao histórico do animal (append-only, sem remoção)
    vacinas[animalId].push(Vacina({
        nome: nomeVacina,
        lote: lote,
        dataAplicacao: dataAplicacao,
        proximaDose: proximaDose,
        veterinario: veterinario,
        clinica: clinica
    }));

    // Emite evento para rastreabilidade e indexação
    emit VacinaRegistrada(animalId, nomeVacina, dataAplicacao);
}
```

Print de um trecho relevante do VacPet.sol (Estrutura de dados, modificadores e função de escrita - registrarVacina)

5. Funções do Contrato

5.1 Funções de Escrita (custam gas)

Função	Gas médio	Controle de acesso	Descrição
cadastrarAnimal()	~130.000	Qualquer endereço	Cadastra animal; auto-ID; tutor = msg.sender
registrarVacina()	~95.000	Apenas tutor do animal	Adiciona vacina ao histórico (append-only)
transferirTutor()	~55.000	Apenas tutor do animal	Transfere tutoria; preserva histórico completo

5.2 Funções de Leitura (gratuitas, sem gas)

Função	Retorno	Descrição
consultarAnimal(uint256)	Animal memory	Dados completos de um animal por ID
consultarVacinas(uint256)	Vacina[] memory	Histórico completo de vacinas de um animal
totalVacinas(uint256)	uint256	Quantidade de vacinas registradas para um animal
animaisDoTutor(address)	uint256[] memory	IDs de todos os animais de um tutor
totalAnimais()	uint256	Total de animais cadastrados no contrato

5.3 Eventos Emitidos

O contrato emite três eventos para permitir indexação off-chain e rastreabilidade completa de todas as operações:

Evento	Parâmetros e Contexto
AnimalCadastrado	(uint256 indexed id, string nome, address indexed tutor) — emitido no cadastro
VacinaRegistrada	(uint256 indexed animalId, string vacina, uint256 data) — emitido no registro
TutorTransferido	(uint256 indexed animalId, address antigo, address novo) — emitido na transferência

5.4 Modificadores de Acesso

Modificador	Comportamento
apenasTutor(animalId)	Reverte com 'VacPet: apenas o tutor pode executar esta acao' se msg.sender != tutor
animalExiste(animalId)	Reverte com 'VacPet: animal nao encontrado ou inativo' se animal.ativo == false

6. Testes Automatizados

6.1 Visão Geral

O projeto possui uma suíte completa de 42 testes automatizados, escritos com Hardhat + Ethers.js v6 + Chai. Os testes cobrem 100% das linhas, branches e funções do contrato, validando fluxos de sucesso, controle de acesso, validações de entrada e integridade dos dados.

Grupo de Testes	Qtd.	O que valida
Deploy	2	Deploy correto, estado inicial zerado
cadastrarAnimal	10	IDs únicos, campos corretos, eventos, vínculos por tutor
registrarVacina	7	Campos da vacina, append-only, controle de acesso, ID inválido
transferirTutor	8	Transferência, preservação de histórico, endereço zero, acesso
consultarAnimal	3	ID inexistente, consulta pública por terceiros
consultarVacinas	3	Array vazio, ID inválido, consulta pública
totalVacinas	2	Contagem correta após inserções
animaisDoTutor	3	Lista correta, isolamento entre tutores
totalAnimais	2	Zero inicial, incremento correto
Cenários de integração	3	Fluxo completo, adoção com histórico, múltiplos atores
TOTAL	42	100% passando

6.2 Cobertura de Código

Relatório de cobertura — solidity-coverage

```

VacPet
Deploy
  ✓ deve fazer deploy com sucesso e expor o endereço do contrato
  ✓ deve inicializar com totalAnimais = 0
cadastrarAnimal
  ✓ deve cadastrar um animal e retornar ID = 1 no primeiro cadastro
  ✓ deve incrementar totalAnimais a cada cadastro
  ✓ deve emitir o evento AnimalCadastrado com os dados corretos
  ✓ deve vincular o animal ao tutor que enviou a transação
  ✓ deve gravar todos os campos do animal corretamente (51ms)
  ✓ deve adicionar o ID do animal na lista animaisDoTutor
  ✓ deve gerar IDs únicos e crescentes para múltiplos animais
  ✓ deve permitir que tutores diferentes cadastrem animais simultaneamente (39ms)
registrarVacina
  ✓ deve registrar uma vacina com sucesso
  ✓ deve gravar todos os campos da vacina corretamente (48ms)
  ✓ deve emitir o evento VacinaRegistrada com os dados corretos
  ✓ deve aceitar proximaDose = 0 (sem próxima dose prevista)
  ✓ deve acumular múltiplas vacinas em ordem (append-only) (45ms)
  ✓ deve rejeitar se chamado por quem não é o tutor
  ✓ deve rejeitar para um animalId inexistente
transferirTutor
  ✓ deve transferir a tutoria para o novo tutor
  ✓ deve adicionar o animal na lista do novo tutor
  ✓ deve preservar o ID do animal na lista do tutor antigo (rastreabilidade)
  ✓ deve emitir o evento TutorTransferido com os endereços corretos
  ✓ deve rejeitar transferência para address(0)
  ✓ deve rejeitar se chamado por quem não é o tutor atual
  ✓ deve rejeitar para animalId inexistente
  ✓ novo tutor deve poder registrar vacinas após a transferência
  ✓ tutor antigo não deve poder registrar vacinas após a transferência
consultarAnimal
  ✓ deve rejeitar consulta de ID inexistente (ID 0 nunca existe)
  ✓ deve rejeitar consulta de ID fora do range cadastrado
  ✓ deve permitir consulta pública (qualquer conta)
consultarVacinas
  ✓ deve retornar array vazio para animal sem vacinas
  ✓ deve rejeitar consulta de ID inexistente
totalVacinas
  ✓ deve retornar 0 para animal sem vacinas registradas
  ✓ deve incrementar a cada vacina registrada
animaisDoTutor
  ✓ deve retornar array vazio para tutor sem animais
  ✓ deve listar todos os animais de um tutor
  ✓ não deve misturar animais de tutores diferentes
totalAnimais
  ✓ deve retornar 0 antes de qualquer cadastro
  ✓ deve refletir o total correto após N cadastros (43ms)
Cenários de integração
  ✓ fluxo completo: cadastro → vacinação → consulta pública (71ms)
  ✓ fluxo completo: adoção com histórico preservado (85ms)
  ✓ fluxo completo: múltiplos animais, múltiplos tutores, múltiplas vacinas (62ms)

42 passing (2s)

```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts\	100	100	100	100	
VacPet.sol	100	100	100	100	
All files	100	100	100	100	

Print do relatório da saída do terminal com `npm run test:coverage`

A cobertura foi gerada com o plugin solidity-coverage e atingiu 100% em todas as dimensões:

Métrica	Resultado	Detalhe
Statements (linhas)	100% — 17/17	Todas as linhas executáveis exercitadas
Branches (desvios)	100% — 18/18	Todos os if/require testados em ambos os ramos
Functions (funções)	100% — 10/10	Todas as funções do contrato chamadas

6.3 Cenários de Integração End-to-End

Três cenários de integração validam fluxos completos do sistema:

- Fluxo completo: cadastro → vacinação múltipla → consulta pública por terceiro sem carteira.
- Fluxo de adoção: cadastro → vacina → transferência de tutor → novo tutor registra vacina → histórico preservado com 2 entradas.
- Fluxo multiator: 2 tutores, 3 animais, 3 vacinas de tipos diferentes — verificação de isolamento e contagens corretas.

Saída do terminal — npm test

```

consultarAnimal
  ✓ deve rejeitar consulta de ID inexistente (ID 0 nunca existe)
  ✓ deve rejeitar consulta de ID fora do range cadastrado
  ✓ deve permitir consulta pública (qualquer conta)
consultarVacinas
  ✓ deve retornar array vazio para animal sem vacinas
  ✓ deve rejeitar consulta de ID inexistente
  ✓ deve permitir consulta pública por qualquer endereço
totalVacinas
  ✓ deve retornar 0 para animal sem vacinas registradas
  ✓ deve incrementar a cada vacina registrada
animaisDoTutor
  ✓ deve retornar array vazio para tutor sem animais
  ✓ deve listar todos os animais de um tutor
  ✓ não deve misturar animais de tutores diferentes
totalAnimais
  ✓ deve retornar 0 antes de qualquer cadastro
  ✓ deve refletir o total correto após N cadastros
Cenários de integração
  ✓ fluxo completo: cadastro → vacinação → consulta pública
  ✓ fluxo completo: adoção com histórico preservado
  ✓ fluxo completo: múltiplos animais, múltiplos tutores, múltiplas vacinas

42 passing (2s)

```

Print do terminal com todos os 42 testes passando (npx hardhat test)

Relatório de gas — npm run test:gas

Solc version: 0.8.20		Optimizer enabled: false		Runs: 200	Block limit: 60000000 gas	
Methods						
Contract	Method	Min	Max	Avg	# calls	usd (avg)
VacPet	cadastrarAnimal	240721	257917	255408	48	-
VacPet	registrarVacina	152116	189224	182454	19	-
VacPet	transferirTutor	-	-	74598	8	-
Deployments					% of limit	
VacPet		-	-	2342322	3.9 %	-

42 passing (2s)

Print da tabela de consumo de gas por função gerada pelo hardhat-gas-reporter

7. Interface Web (Frontend)

7.1 Visão Geral

O frontend é um arquivo HTML único e autocontido (sem frameworks, sem bundler, sem dependências instaladas). A comunicação com o contrato é feita via Ethers.js carregado por CDN. O MetaMask é usado como provedor Web3 para autenticação e assinatura de transações.

7.2 Funcionalidades da Interface

Funcionalidade	Descrição
Conectar carteira	Conecta MetaMask à rede Sepolia; exibe endereço da conta
Cadastrar animal	Formulário com nome, espécie, raça, cor e data de nascimento
Registrar vacina	Vincula vacina (nome, lote, datas, veterinário, clínica) a um animal por ID
Consultar animal	Busca pública por ID — exibe todos os dados do registro
Consultar vacinas	Busca pública por ID do animal — exibe histórico completo
Transferir tutoria	Transfere o animal para um novo endereço de carteira

7.3 Capturas de Tela do Sistema

Tela principal — conexão com MetaMask



Formulário de cadastro de animal

Cadastrar novo animal

NOME DO ANIMAL: Coxinha

ESPÉCIE: Cão

RAÇA: Golden

COR / PELAGEM: Amarela

DATA DE NASCIMENTO: 10/10/2025

Registrar na Blockchain

✓ Animal cadastrado com sucesso! ID: 9
[Ver na Etherscan](#)

Print do formulário de cadastro preenchido e a transação confirmada no MetaMask

Registro de vacina

Registrar vacina

ID DO ANIMAL: 9

NOME DA VACINA: Antirrábica

NÚMERO DO LOTE: LOT2026-01

DATA DE APLICAÇÃO: 09/06/2026

PRÓXIMA DOSE (OPCIONAL): 09/06/2027

VETERINÁRIO: DR. Djairo CRMV 12345

CLÍNICA / HOSPITAL: Clínica Pet Saudável

Registrar na Blockchain

✓ Vacina registrada com sucesso! ID: 9

Print do formulário de registro de vacina

Consulta pública do histórico vacinal



Print da consulta de vacinas por ID do animal, mostrando o histórico completo retornado

7.4 Decisões de Design do Frontend

- Sem framework: HTML + CSS + JS puro reduz a superfície de ataque e elimina dependências de build;
- Ethers.js via CDN: sem necessidade de npm install no ambiente do usuário;
- Dados sensíveis nunca enviados: o frontend não coleta nem transmite CPF ou endereço físico;
- Consulta sem carteira: as funções de leitura funcionam mesmo sem MetaMask conectado.

8. Stack Tecnológica e Ferramentas

Ferramenta / Biblioteca	Versão	Papel no Projeto
Solidity	^0.8.20	Linguagem do smart contract
Hardhat	^2.22.17	Compilação, rede local, deploy e orquestração de testes
Ethers.js	^6.7.0	Comunicação com contrato (frontend e testes)
Chai	^4.3.7	Framework de asserções nos testes
hardhat-gas-reporter	^1.0.8	Relatório de consumo de gas por função
solidity-coverage	^0.8.1	Relatório de cobertura de código (statements/branches/fns)
cross-env	^7.0.3	Compatibilidade de variáveis de ambiente (Windows/Linux)
dotenv	^17.4.2	Carregamento seguro de variáveis sensíveis (.env)
MetaMask	Extensão browser	Carteira Web3 do usuário final
Sepolia Testnet	Rede Ethereum	Ambiente de testes públicos com ETH de faucet
Node.js	>= 18	Ambiente de execução dos scripts

8.1 Scripts NPM Disponíveis

Comando	Ação
npm test	Executa todos os 42 testes na rede local Hardhat
npm run test:coverage	Gera relatório HTML de cobertura de código em coverage/index.html
npm run test:gas	Executa testes com relatório de consumo de gas (cross-env REPORT_GAS=true)
npx hardhat compile	Compila o contrato Solidity com verificação de tipos
npx hardhat run scripts/deploy.js --network sepolia	Faz deploy do contrato na Sepolia Testnet
npx http-server frontend -p 3000	Serve o frontend localmente na porta 3000

9. Análise de Segurança

9.1 Controle de Acesso

O contrato implementa controle de acesso granular no nível de tutoria individual. Não existe um 'admin' global — cada tutor controla apenas seus próprios animais:

- Somente o tutor de um animal pode registrar vacinas nele;
- Somente o tutor pode transferir a tutoria para outro endereço;
- Qualquer pessoa pode consultar dados (leitura pública);
- A transferência para `address(0)` é explicitamente bloqueada.

9.2 Integridade dos Dados

- Histórico de vacinas é `append-only`: vacinas nunca são removidas ou alteradas após registro;
- O campo ativo permite desativar um registro sem apagar o histórico (auditabilidade preservada);
- O timestamp criado em usa `block.timestamp` — prova criptográfica de existência on-chain;
- IDs começam em 1: ID 0 nunca é válido, simplificando verificações de existência.

9.3 Privacidade por Design

- Dados sensíveis (CPF, endereço físico) nunca são coletados nem armazenados, nem on-chain, nem off-chain.
- Apenas o endereço da carteira (wallet) do tutor é registrado, não há PII vinculada obrigatoriamente;
- A interface web não tem back-end próprio: não há servidor que colete dados dos usuários.

10. Deploy e Configuração

10.1 Pré-Requisitos

Requisito	Detalhe
Node.js	>= 18 LTS
MetaMask	Extensão instalada no navegador
ETH Sepolia (testnet)	Obtido via faucet (sepoliafaucet.com)
SEPOLIA_RPC_URL	URL RPC da Sepolia (ex: Alchemy, Infura)
PRIVATE_KEY	64 hex chars da conta deployer (sem 0x)

10.2 Passo a Passo de Deploy

1. Clonar repositório e instalar dependências:

```
git clone https://github.com/lielsonandrade/vacpet
cd vacpet && npm install
```

2. Configurar variáveis de ambiente:

```
cp .env.example .env
# Editar .env com PRIVATE_KEY e SEPOLIA_RPC_URL
```

3. Compilar o contrato:

```
npx hardhat compile
```

4. Fazer deploy na Sepolia:

```
npx hardhat run scripts/deploy.js --network sepolia
```

5. Atualizar o frontend com o endereço gerado:

```
# Copiar endereço do contrato e colar em frontend/index.html
# na variável CONTRACT_ADDRESS
```

6. Servir o frontend localmente:

```
npx http-server frontend -p 3000
```

10.3 Endereço Deployado na Sepolia

Rede	Endereço do Contrato
Sepolia Testnet	0x87674376e5eE07ECe76D697446Fa25617214E9F4

11. Conclusão e Trabalhos Futuros

Conclusão

O VacPet demonstra com sucesso a aplicação prática de tecnologia blockchain para resolver um problema real e cotidiano: a gestão confiável do histórico de saúde animal. A solução é tecnicamente sólida, com 100% de cobertura de testes, controle de acesso correto e uma arquitetura simples e auditável.

A decisão de não armazenar dados sensíveis on-chain, combinada com a consulta pública irrestrita, equilibra privacidade e transparência de forma adequada. O design append-only do histórico de vacinas garante que nenhum registro possa ser apagado ou adulterado retroativamente.